MongoDB

What you will learn:

✔ What MongoDB is and why it exists
✔ How to create, read, update and delete data
✔ How to search and filter data fast
✔ How to summarise data with the Aggregation Pipeline
✔ How to use MongoDB in real data engineering projects

Chapter 1: What is MongoDB?

what problem does MongoDB solve?

Imagine you work at an online store. Every customer order looks a little different:

- Some customers add a gift message.
- Some orders have 1 item, others have 20.
- International orders have extra fields like customs info.

In a traditional database (like MySQL or Postgres), every row must have exactly the same columns. You would need to add a column for gift_message, leave it empty for 90% of orders, and keep changing your table every time a new field appears. That gets messy fast.

Think of it like this...

A traditional (SQL) database is like a spreadsheet — every row must fill the same columns.

MongoDB is like a folder of sticky notes — each note can have different info written on it.

You don't need every note to look the same. You just drop it in the folder.

So what IS MongoDB?

MongoDB is a database that stores data as documents — flexible, JSON-like objects that can have any shape you need. It is called a NoSQL database because it does not use the traditional table-and-row structure of SQL databases.

MongoDB stores data as documents (like JSON)

Here is what a MongoDB document looks like:

```
{
  "name": "Alice",
  "age": 29,
  "city": "Atlanta",
  "skills": ["Python", "MongoDB", "SQL"]
}
```

Notice: it has a list inside (skills). SQL cannot do that natively. MongoDB can.

MongoDB vs SQL — Side by Side

If you have used Excel or SQL before, this table will help you map the concepts:

SQL Term	MongoDB Equivalent
Database	Database (same name!)
Table	Collection
Row	Document
Column	Field
Primary Key	_id (auto-created by MongoDB)
JOIN	$lookup (inside the Aggregation Pipeline)
SELECT / WHERE	find() with filters
GROUP BY	$group stage in the Aggregation Pipeline

When should you use MongoDB?

✔ MongoDB is a great fit when...

Your data has different shapes (e.g. product catalogs, user profiles, IoT sensor data)

You need to store nested data — lists inside lists, objects inside objects

You need to ingest data very quickly (it is extremely fast at writes)

You are building data pipelines that process semi-structured data like JSON or API responses

⚠ Watch Out!

MongoDB is NOT always the best choice.

If your data is highly relational (lots of strict relationships between tables),

a SQL database like Postgres may be better.

Good engineers pick the right tool for the job.

Chapter 2: Architecture — How MongoDB is Organised

The three levels you need to know

MongoDB organises data in three levels. Think of it like a filing cabinet:

Level	What it is
Database	The filing cabinet itself. You can have many databases on one server.
Collection	A drawer inside the cabinet. Groups related documents together.
Document	A single file inside the drawer. This is where your actual data lives.

Think of it like this...

DATABASE = a Library building

COLLECTION = a Bookshelf in that library

DOCUMENT = a single Book on the shelf

You walk into the library (database), go to the right shelf (collection),

and pick up the book you need (document).

What does a Document look like?

A document is just a JSON object. Here is a real example — a pipeline run event stored in a data engineering system:

```
{
  "_id"     : ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"),  // auto-created unique ID
  "name"    : "orders_etl",                          // pipeline name
  "status"  : "success",                             // did it work?
  "records" : 14500,                                 // how many rows pro-
cessed
  "date"    : ISODate("2025-06-01T09:00:00Z"),       // when it ran
  "errors"  : []                                     // list of errors (empty
= none)

}
```

Notice the _id field at the top. MongoDB creates this automatically for every document. It is guaranteed to be unique — you never need to create it yourself.

The key MongoDB programs

When you install MongoDB, you get a few programs. Here is what each one does:

Program	What it does
mongod	The actual database server. This must be running at all times.
mongosh	The shell (command line) where you type queries. Like a SQL client.
mongos	A router used only in very large setups. Ignore this for now.

Chapter 3: Installation — Get MongoDB Running

We will use Docker. It is the simplest, safest way to run MongoDB — no risk of messing up your computer, and it works the same on Windows, Mac and Linux.

Why Docker?

Docker runs MongoDB inside an isolated 'container'.

You don't have to configure anything. One command and you are done.

When you are finished, you can delete it cleanly.

STEP 1 Install Docker Desktop

Go to: https://www.docker.com/products/docker-desktop

Download and install for your operating system (Windows / Mac / Linux).

After installing, open Docker Desktop and wait for it to say 'Engine running'.

STEP 2 Start MongoDB with one command

Open your terminal (Command Prompt on Windows, Terminal on Mac/Linux) and paste this:

```
docker run -d \
  --name my-mongodb \
  -p 27017:27017 \
  mongo:7.0
```

What does this do?

- -d → runs in the background (you get your terminal back)
- --name my-mongodb → gives it a friendly name
- -p 27017:27017 → opens port 27017 so you can connect to it
- mongo:7.0 → uses MongoDB version 7

STEP 3 Connect to MongoDB using the shell

Now open the MongoDB shell inside the container:

```
docker exec -it my-mongodb mongosh
```

You should see something like:

```
Current Mongosh Log ID: ...
Connecting to: mongodb://127.0.0.1:27017
test>
```

The `test>` prompt means you are connected and ready to go!

STEP 4 Run your first commands

Type these inside the mongosh shell to verify everything works:

```
// See all databases
show dbs

// Switch to (or create) a database called 'school'
use school

// Insert your first document
db.students.insertOne({ name: 'Alice', age: 22, course: 'Data Engineering' })

// Read it back
db.students.find()
```

STEP 5 Install pymongo (Python driver)

To use MongoDB from Python, install the pymongo library:

```
pip install pymongo
```

Test it works — create a file called test_mongo.py and run it:

```
from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient('mongodb://localhost:27017/')
db = client['school']

# Insert a document
db.students.insert_one({ 'name': 'Bob', 'age': 23 })

# Print all students
for student in db.students.find():
    print(student)
```

Chapter 4: CRUD Operations

CRUD stands for Create, Read, Update, Delete. These are the four things you can do with any piece of data. Let's go through each one with simple examples.

> **Think of it like this...**
>
> CRUD is like managing contacts on your phone:
>
> CREATE = Add a new contact
>
> READ = Look up a contact
>
> UPDATE = Edit a contact's number
>
> DELETE = Remove a contact

Setup — Our Sample Data

We will use a collection of pipeline runs throughout this chapter. Paste this into mongosh to create the data:

```
use de_practice


db.runs.insertMany([
  { name: 'orders_etl',   status: 'success', records: 14500, duration: 32,
env: 'prod' },
  { name: 'users_sync',   status: 'failed',  records: 0,     duration: 5,
env: 'prod' },
  { name: 'stock_etl',    status: 'success', records: 8900,  duration: 18,
env: 'prod' },
  { name: 'orders_etl',   status: 'success', records: 15200, duration: 35,
env: 'prod' },
  { name: 'test_pipeline',status: 'success', records: 100,   duration: 2,
env: 'dev'  },
  { name: 'users_sync',   status: 'success', records: 3210,  duration: 9,
env: 'prod' },
])
```

CREATE — Adding data

insertOne() — Add one document

Use this when you want to add a single record.

```
db.runs.insertOne({
  name: 'payments_etl',
  status: 'success',
  records: 5000,
  duration: 12,
  env: 'prod'
})

// MongoDB replies with:
// { acknowledged: true, insertedId: ObjectId('...') }
```

insertMany() — Add multiple documents at once

Much faster than calling insertOne() in a loop. Always prefer this for bulk inserts.

```
db.runs.insertMany([
  { name: 'job_a', status: 'success', records: 1000, duration: 4, env:
'dev' },
  { name: 'job_b', status: 'failed',  records: 0,    duration: 1, env: 'dev' }
])
```

READ — Fetching data

find() — Get all documents

```
db.runs.find()   // returns everything in the collection
```

find({ filter }) — Get only matching documents

Put your search conditions inside the curly braces:

```
// Only successful runs
db.runs.find({ status: 'success' })

// Only failed prod runs
db.runs.find({ status: 'failed', env: 'prod' })

// Records greater than 10,000  ($gt means 'greater than')
db.runs.find({ records: { $gt: 10000 } })

// Duration between 10 and 30 seconds
db.runs.find({ duration: { $gte: 10, $lte: 30 } })
```

Choose which fields to show

The second argument to find() controls which fields appear in results. 1 = show, 0 = hide.

```
// Show only name and status, hide _id
db.runs.find({ env: 'prod' }, { name: 1, status: 1, _id: 0 })
```

Sort, Limit, Count

```
// Sort by records, largest first
db.runs.find().sort({ records: -1 })

// Show only the top 3 results
db.runs.find().sort({ records: -1 }).limit(3)

// Count how many successful runs exist
db.runs.countDocuments({ status: 'success' })
```

Filter Operators

$gt — greater than { records: { $gt: 5000 } }
$gte — greater than or equal { duration: { $gte: 10 } }
$lt — less than { records: { $lt: 1000 } }
$lte — less than or equal { duration: { $lte: 30 } }
$ne — not equal { status: { $ne: 'failed' } }
$in — value is in this list { env: { $in: ['prod', 'staging'] } }

UPDATE — Changing data

updateOne() — Change one document

Two arguments: (1) which document to find, (2) what to change.

```
// Fix the failed users_sync run — it actually succeeded with 3100 records
db.runs.updateOne(
  { name: 'users_sync', status: 'failed' },    // FIND this document
  { $set: { status: 'success', records: 3100 } }  // CHANGE these fields
)
```

updateMany() — Change all matching documents

```
// Mark all dev runs as archived
db.runs.updateMany(
```

```
  { env: 'dev' },
  { $set: { archived: true } }
)
```

🛠 Common Update Operators

$set — Set a field to a new value

$inc — Add a number to a field e.g. { $inc: { records: 500 } }

$unset — Delete a field from the document e.g. { $unset: { archived: '' } }

$push — Add an item to an array e.g. { $push: { tags: 'important' } }

DELETE — Removing data

```
// Delete one specific document
db.runs.deleteOne({ name: 'test_pipeline', env: 'dev' })

// Delete ALL failed runs
db.runs.deleteMany({ status: 'failed' })
```

⚠ Watch Out!

deleteMany() with a wrong filter can delete a lot of data at once.

Always run find() with the same filter FIRST to see what will be deleted.

For example: db.runs.find({ status: 'failed' }) — check the results, then delete.

✏ Try It Yourself

Using the runs collection:

1. Find all runs where records > 8000.
2. Find only the name and status of prod runs (hide _id).
3. Update the stock_etl run — change its duration to 20.
4. Add a field 'reviewed: true' to all successful prod runs using updateMany().
5. Delete all dev runs.
6. Run countDocuments() to confirm they are gone.

Chapter 5: Indexing — Making Queries Fast

The problem without an index

Imagine your runs collection has 10 million documents. When you run:

```
db.runs.find({ name: 'orders_etl' })
```

Without an index, MongoDB reads every single one of the 10 million documents to find matches. This is called a Collection Scan — it is slow and expensive.

Think of it like this...

Imagine finding a name in a phone book that is NOT alphabetically sorted.

You would have to read every page from start to finish.

An index is like the alphabetical sorting in a phone book.

You jump straight to the right section and find the name instantly.

Creating an Index

```
// Create an index on the 'name' field
db.runs.createIndex({ name: 1 })   // 1 = ascending order

// Create an index on 'status'
db.runs.createIndex({ status: 1 })

// Compound index — covers queries that filter by BOTH env AND status
db.runs.createIndex({ env: 1, status: 1 })

// See all existing indexes
db.runs.getIndexes()

// Remove an index you no longer need
db.runs.dropIndex({ name: 1 })
```

Checking if your index is being used

Use explain() to see what MongoDB is doing behind the scenes:

```
db.runs.find({ name: 'orders_etl' }).explain('executionStats')
```

Look for these two things in the output:

What you see	What it means
stage: 'COLLSCAN'	BAD — MongoDB read every document. You need an index.
stage: 'IXSCAN'	GOOD — MongoDB used an index. Query is fast.
totalDocsExamined: 10000000	BAD — Scanned 10 million docs to return a few.
totalDocsExamined: 2	GOOD — Only looked at 2 docs to get the answer.

✏ Try It Yourself

1. Run explain() on db.runs.find({ status: 'success' }) before creating any index.
   Note the stage value — it should say COLLSCAN.
2. Create an index: db.runs.createIndex({ status: 1 })
3. Run the same explain() again.
   The stage should now say IXSCAN — the index is being used!

Chapter 6: The Aggregation Pipeline

What is the Aggregation Pipeline?

The Aggregation Pipeline is MongoDB's most powerful feature. It lets you analyse and transform your data — count things, calculate averages, group data, and more.

Think of it like this...

Think of it like an assembly line in a factory.

Raw materials (your documents) enter at one end.

They pass through several stations (stages), each one doing something to them.

A finished product (your results) comes out the other end.

Each stage does one job — filter, group, sort, reshape, etc.

The Basic Structure

```
db.collection.aggregate([
  { $stage1 },   // first transformation
  { $stage2 },   // second transformation
  { $stage3 }    // final output
])
```

The stages run in order — the output of stage 1 becomes the input of stage 2, and so on.

Stage 1: $match — Filter documents

$match is like the WHERE clause in SQL. Use it FIRST to reduce the number of documents being processed.

```
db.runs.aggregate([
  { $match: { status: 'success', env: 'prod' } }
  // Only successful prod runs pass through to the next stage
])
```

Stage 2: $group — Summarise data

$group is like GROUP BY in SQL. You pick a field to group by, then calculate things for each group.

```
db.runs.aggregate([
  { $match: { status: 'success' } },

  { $group: {
    _id: '$name',                        // group by pipeline name
    total_records: { $sum: '$records' },  // add up all records
    how_many_runs: { $sum: 1 },           // count the runs
    avg_duration:  { $avg: '$duration' }  // average duration
  }}
])

// Example output:
// { _id: 'orders_etl',  total_records: 29700, how_many_runs: 2, avg_duration: 33.5 }
// { _id: 'users_sync',  total_records: 3210,  how_many_runs: 1, avg_duration: 9    }
// { _id: 'stock_etl',   total_records: 8900,  how_many_runs: 1, avg_duration: 18   }
```

$group Operators

$sum — Add up values (use $sum: 1 to count documents)

$avg — Calculate the average

$min — Find the smallest value

$max — Find the largest value

$push — Collect all values into an array

Stage 3: $sort — Order the results

```
db.runs.aggregate([
  { $match: { status: 'success' } },
  { $group: { _id: '$name', total_records: { $sum: '$records' } }},
  { $sort: { total_records: -1 } }   // -1 = largest first, 1 = smallest first
])
```

Stage 4: $project — Reshape the output

$project lets you choose which fields appear and rename them to something readable.

```
db.runs.aggregate([
  { $match: { env: 'prod' } },
  { $project: {
    _id: 0,                              // hide the _id field
```

```
    pipeline: '$name',                  // rename 'name' to 'pipeline'
    status: 1,                          // keep status
    records_in_thousands: { $divide: ['$records', 1000] }  // compute a new
field
  }}
])
```

Putting it all together — A complete example

Goal: Find the top 3 pipelines by total records processed, for prod environments only.

```
db.runs.aggregate([
  // Step 1: Keep only prod runs
  { $match: { env: 'prod' } },

  // Step 2: Group by pipeline name and sum the records
  { $group: {
    _id: '$name',
    total_records: { $sum: '$records' },
    run_count: { $sum: 1 }
  }},

  // Step 3: Sort by total records, largest first
  { $sort: { total_records: -1 } },

  // Step 4: Only show the top 3
  { $limit: 3 },

  // Step 5: Clean up the field names
  { $project: { _id: 0, pipeline: '$_id', total_records: 1, run_count: 1 } }
])

// Output:
// { pipeline: 'orders_etl', total_records: 29700, run_count: 2 }
// { pipeline: 'stock_etl',  total_records: 8900,  run_count: 1 }
// { pipeline: 'users_sync', total_records: 3210,  run_count: 1 }
```

✏ Try It Yourself

Try these aggregation challenges on the runs collection:

1. EASY: Count how many runs there are for each status value (success / failed).
   Hint: $group on '$status', use $sum: 1 to count.

2. MEDIUM: Find the pipeline with the highest average duration.
   Hint: $group on '$name', use $avg on '$duration', then $sort.

3. HARD: For prod runs only, calculate the success rate (%) per pipeline.
   Hint: Use $group with $sum: 1 for total, and $sum with $cond to count successes.

Chapter 7: MongoDB in Data Engineering Pipelines

How MongoDB fits into a real DE pipeline

In a data engineering job, MongoDB is rarely used alone. It plugs into a bigger system. Here are the most common ways:

Role in Pipeline	Example
Raw Data Landing Zone	API data, webhook events, or IoT sensor readings arrive and are stored in MongoDB first.
Operational Data Store	Cleaned data lives here before being loaded to a data warehouse.
Transformation Engine	Use the Aggregation Pipeline to group, filter, and reshape data in-database.
Source for Spark	Spark reads MongoDB collections as DataFrames for large-scale processing.
Real-time CDC	Change Streams detect new/changed documents and trigger downstream jobs.

Using MongoDB from Python

In real DE work, you write Python scripts instead of using the shell. Here is a complete, working example:

```
from pymongo import MongoClient
from datetime import datetime

# --- Step 1: Connect ---
client = MongoClient('mongodb://localhost:27017/')
db = client['de_practice']
runs = db['runs']

# --- Step 2: Insert data ---
runs.insert_one({
    'name': 'python_etl_job',
    'status': 'success',
    'records': 25000,
    'duration': 60,
    'env': 'prod',
    'timestamp': datetime.utcnow()
})

# --- Step 3: Query ---
```

```
print('All successful runs:')
for doc in runs.find({ 'status': 'success' }, { '_id': 0, 'name': 1,
'records': 1 }):
    print(' -', doc['name'], ':', doc['records'], 'records')

# --- Step 4: Run an aggregation ---
pipeline = [
    { '$match': { 'env': 'prod' } },
    { '$group': { '_id': '$name', 'total': { '$sum': '$records' } } },
    { '$sort': { 'total': -1 } }
]
print('\nTotal records per pipeline:')
for r in runs.aggregate(pipeline):
    print(f"  {r['_id']:20s}  {r['total']:,} records")
```

Mini Project: Build a Pipeline Monitor

Project: Pipeline Health Dashboard

Goal: Build a Python script that:

1. Inserts 20 fake pipeline run documents with random data
2. Calculates success rate per pipeline
3. Finds the slowest pipeline (highest avg duration)
4. Prints a clean summary report

```
import random
from pymongo import MongoClient
from datetime import datetime, timedelta

client = MongoClient('mongodb://localhost:27017/')
db = client['de_project']
db.runs.drop()  # fresh start

# --- Generate fake data ---
pipelines = ['orders_etl', 'users_sync', 'stock_etl', 'payments_etl']
statuses  = ['success', 'success', 'success', 'failed']  # 75% success rate

docs = []
for i in range(20):
    docs.append({
```

```
        'name':     random.choice(pipelines),
        'status':   random.choice(statuses),
        'records':  random.randint(100, 50000),
        'duration': random.randint(2, 120),
        'env':      'prod',
        'date':     datetime.utcnow() - timedelta(days=random.randint(0,30))
    })
db.runs.insert_many(docs)
print(f'Inserted {len(docs)} documents.')

# --- Aggregation: success rate per pipeline ---
pipeline = [
    { '$group': {
        '_id': '$name',
        'total':    { '$sum': 1 },
        'success':  { '$sum': { '$cond': [{ '$eq': ['$status','success'] }, 1,
0] } },
        'avg_dur':  { '$avg': '$duration' }
    }},
    { '$sort': { '_id': 1 } }
]

print('\n=== Pipeline Health Report ===')
print(f'{"Pipeline":<20} {"Runs":>5} {"Success%":>9} {"Avg Duration":>13}')
print('-' * 52)
for r in db.runs.aggregate(pipeline):
    rate = round(r['success'] / r['total'] * 100)
    print(f"{r['_id']:<20} {r['total']:>5} {rate:>8}%
{r['avg_dur']:>10.1f}s")
```

Quick Reference Card

Most Used Commands

```
// === NAVIGATE ===
show dbs                          // list databases
use mydb                          // switch to a database
show collections                  // list collections in current db

// === CREATE ===
db.col.insertOne({ ... })
db.col.insertMany([ {...}, {...} ])

// === READ ===
db.col.find()                     // all documents
db.col.find({ field: value })     // with filter
db.col.find({}, { field: 1, _id: 0 })  // with projection
db.col.find().sort({ field: -1 }).limit(5)
db.col.countDocuments({ ... })
db.col.findOne({ ... })

// === UPDATE ===
db.col.updateOne({ filter }, { $set: { ... } })
db.col.updateMany({ filter }, { $set: { ... } })

// === DELETE ===
db.col.deleteOne({ filter })
db.col.deleteMany({ filter })

// === INDEXES ===
db.col.createIndex({ field: 1 })
db.col.getIndexes()

// === AGGREGATION ===
db.col.aggregate([
  { $match: { ... } },
  { $group: { _id: '$field', total: { $sum: '$other' } } },
  { $sort: { total: -1 } },
  { $limit: 10 }
])
```

What to study next

explore these topics:

1. Data Modeling — When to embed vs. reference documents
2. Change Streams — Get notified in real time when data changes
3. MongoDB Atlas — The cloud-hosted version (free tier available)
4. Spark + MongoDB Connector — Process millions of documents with Spark
5. Time Series Collections — Optimised storage for sensor / metric data